

Universidade Federal de Pernambuco
Centro de Tecnologia e Geociências
Programa de Pós-Graduação em Engenharia Elétrica

Barramento Avalon-MM para Wishbone¹

Sumário

1	Introdução	1
2	O Barramento Wishbone	1
2.1	A Interface	1
2.2	O mapeamento dos sinal	2
2.3	Comparando com Barramento Avalon-Memory-Mapped	3
2.3.1	Uma Av-MM-to-Wishbone Brige	3
3	Criando um <i>IP core</i> no Platform Designer	4

1 Introdução

Nas últimas aulas temos trabalhado com uma Dual-Port-RAM para fazer uma ponte ente o HPS e o módulo criado por nós na FPGA. Embora funcional, como estamos até então utilizando um *IP core* da Altera para gerar a Dual-Port-RAM, ficamos limitados as configurações disponíveis para este módulo. O que veremos nesta aula será como construir uma interface padrão para podermos adaptar para “qualquer módulo” que vinhamos a desenvolver na FPGA. Vamos utilizar isto para introduzir duas coisas:

- O barramento Wishbone;
- Como criar um *IP core* no Platform Designer/QSyS/SOPC.

2 O Barramento Wishbone

O barramento Wishbone é um barramento *open-source* mantido pela **Free and Open Source Silicon Foundation** https://wishbone-interconnect.readthedocs.io/_/downloads/en/latest/pdf/ [1, 2]. Ele foi pensado e projetado para ser um padrão aberto para interconexão de *IP cores* em System-on-Chips [1, 2].

2.1 A Interface

No barramento Wishbone existem dois tipos de interface: MASTER e SLAVE. A ligação entre dispositivos MASTER e SLAVE pode ser:

- ***Point-to-Point*** (ponto-a-ponto), mostrada na Figura 1a;

¹Documento em Construção: última atualização às 07:45 de 23 de abril de 2025.

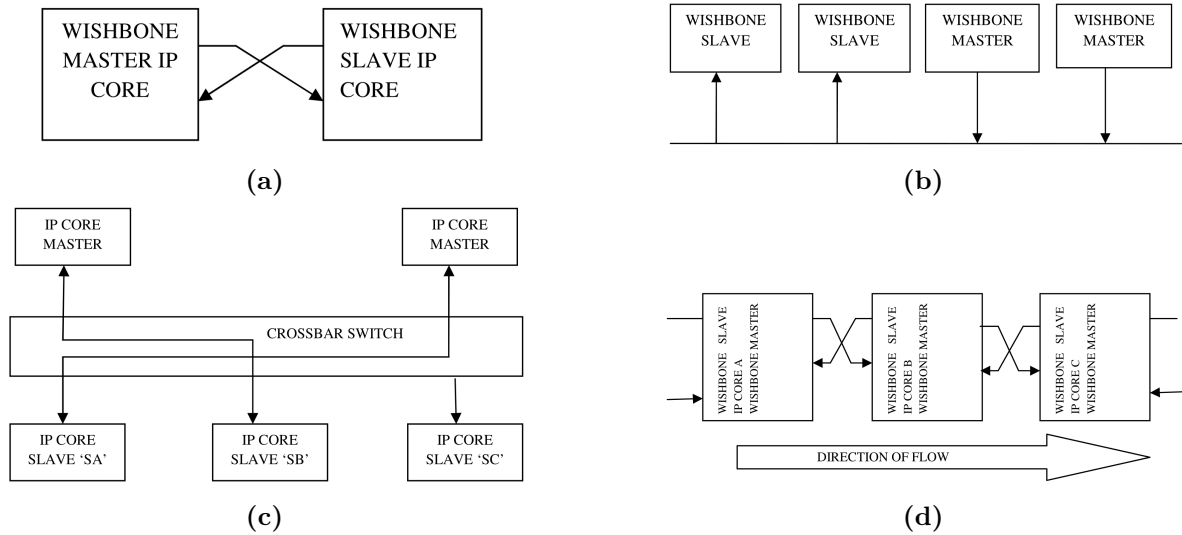


Figura 1: a) Ponto-a-Ponto, b) Barramento Compartilhado, c) Comutador Transversal, d) Fluxo de Dados. Fonte: [2].

- **Shared Bus** (barramento compartilhado), mostrada na Figura 1b. Esta interconexão possibilita ter vários dispositivos MASTER, mas apenas um podendo comunicar por vez.
- **Crossbar Switch** (comutador transversal), mostrada na Figura 1c. Esta interconexão possibilita um ou mais dispositivos MASTER poderem se comunicar com dispositivos SLAVE ao mesmo tempo, desde que não seja o mesmo dispositivo SLAVE.
- **Data Flow** (fluxo de dados), mostrado na Figura 1d. Nesta interconexão, todos os módulos são dotados de uma interface MASTER e uma SLAVE. Os módulos são ligados em sequência de forma que o sinal passará por todos os módulos. Esse tipo de ligação também é chamada de *pipeline* e no caso do Wishbone, suporta ligações em paralelo.

2.2 O mapeamento dos sinal

Na Figura 2 são mostrados os sinais que compõem o barramento Wishbone, se são obrigatórios e sua função.

Signal	Name	Optional	Description
Acknowledged Out	ACK_O	No	Acknowledged signal from the Master to Slave
Address Out/In	ADR_O/I()	No	Address Array
Clock In	CLK_I	No	System Clock for Wishbone Interface.
Error In/Out	ERR_I/O	Yes	Indicates an abnormal cycle termination occurred.
Data In/Out	DAT_I/O	No	Data input/output array, used to send/receive data.
Write Enable In	WE_I	No	Read or Write Signals. If asserted it is Write signal otherwise Read Signal.
Strobe In	STB_I	No	Indicates that the slave is selected The slave asserts either ACK_O, ERR_O
Strobe Out	STB_O	No	Handshaking Signal.
Address tag Out	TGA_O()	Yes	Contains Information about the address array.
Cycle tag type Out	TGC_O()	Yes	Contains Information about the transfer cycle.
Write Enable Out	WE_O	No	Shows if the transfer cycle is a Read or Write cycle.
Reset	RST_I	No	Reset signal

Figura 2: Lista dos sinais da interface Wishbone MASTER e suas funções. Fonte [2].

2.3 Comparando com Barramento Avalon-Memory-Mapped

Os sinais e modo de funcionamentos do barramento Avalon-Memory-Mapped (Av-MM) são muito semelhantes aos do barramento Wishbone [2]. Na Tabela 1 é possível ver a correspondência entre a maioria dos sinais não opcionais.

Tabela 1: Equivalência dos sinais do barramento Av-MM com o Wishbone SLAVE [3].

Av-MM	Wishbone	Direção	Descrição
clk	clk_i	in	sinal de sincronismo
rst	rst_i	in	sinal de reset
chipselct	stb_i	in	ignora o barramento caso <code>chipselct = 0</code>
address	adr_i	in	endereço de escrita ou leitura
readdata	data_o	out	sinal de leitura
write	we_i	in	escreve <code>writedata</code> no <code>address</code>
writedata	data_i	in	sinal de escrita
byteenable	sel_i	in	seleção de bytes dentro da palavra

2.3.1 Uma Av-MM-to-Wishbone Brige

Como o barramento Wishbone é aberto, é interessante que os *IP cores* que vamos desenvolver ao longo da disciplina sejam compatíveis com o barramento Wishbone. No entanto, como as FPGAs que estamos utilizando são da Altera/Intel, é necessária uma tradução entre os sinais do Av-MM para o padrão Wishbone. Uma implementação desta ponte é mostrada no código a seguir (e disponibilizado no github da turma em AULA06):

```

1 //=====
2 // Title: Avalon Memory Mapped to Wishbone Bridge
3 // Description: Wait request is not implemented yet!!
4 //=====
5
6 module avalon_to_wishbone_bridge
7     #(
8         parameter BUS_WIDTH = 5,
9         parameter DATA_WIDTH = 32,
10        parameter BE_WIDTH = 4
11    )
12    (
13        //Common signal:
14        input clk_i,
15        input rst_i,
16        //Avalon MM interface:
17        input avmm_chipselect, //The System Interconnect Fabric has to generate
18        this signal
19        input [BUS_WIDTH-1:0] avmm_address,
20        input avmm_read,
21        output [DATA_WIDTH-1:0] avmm_readdata,
22        input avmm_write,
23        input [DATA_WIDTH-1:0] avmm_writedata,
24        input [BE_WIDTH-1: 0] avmm_byteenable,
25        input avmm_begintransfer, //Is this necessary?
26        output avmm_waitrequest, //It is not implemented yet
27        //Wishbone interface:
28        output [BUS_WIDTH-1:0] adr_o, //Address Out
29        input [DATA_WIDTH-1:0] data_i, //Data In
30        output [DATA_WIDTH-1:0] data_o, //Data Out
31        output we_o, //Write Enable Out
32        output [BE_WIDTH-1: 0] sel_o, //Select output array
33        output stb_o, //Strobe Out
34        input ack_i //Acknowledged Out
35    );
36
37    assign adr_o = avmm_address;
38    assign data_o = avmm_writedata;
39    assign stb_o = avmm_chipselect;
40    assign we_o = avmm_write;
41    assign sel_o = avmm_byteenable;
42
43    assign avmm_readdata = data_i;
44
45 endmodule

```

A ideia é que esse arquivo sirva como uma ponte entre o HPS e os módulos desenvolvidos na FPGA. Para isto, vamos adicionar esta interface como um *IP core* no Platform Designer.

3 Criando um *IP core* no Platform Designer

Neste seção será mostrado o passo-a-passo para criar um *IP core* no Platform Designer para que ele possa ser incluído nos projetos. Como exemplo, utilizaremos o arquivo “avalon_to_wishbone_bridge.sv”, mostrado na Seção 2.3.1.

Ao passo-a-passo é:

1. Abrir o **Platform Designer**, selecionar o arquivo “.qsys” correspondente e clicar em **New** na Aba **IP Catalog**, como destacado na Figura 3.

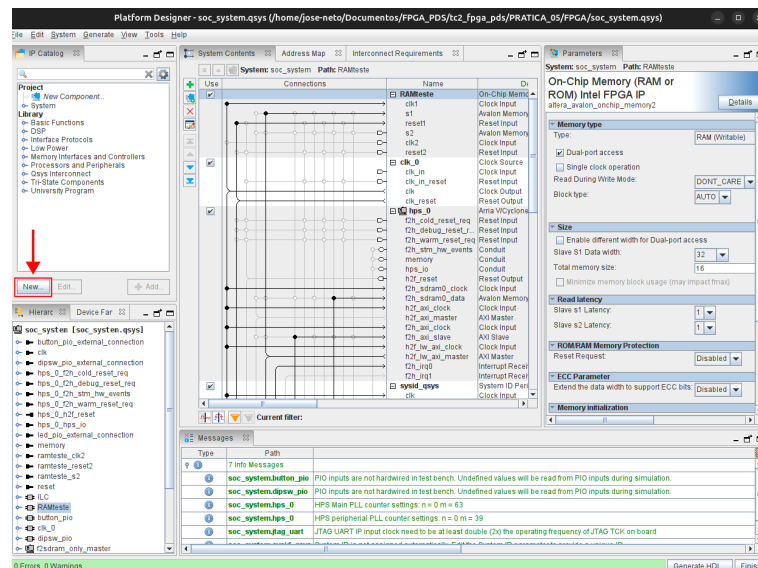


Figura 3: Tela do Platform Designer destacando onde selecionar para criar um novo componente.

2. Será aberta a janela **Component Editor** na aba **Component Type**, da qual parte é mostrada na Figura 4. Os seguintes campos devem ser preenchidos:

- **Name:** avmm_to_wishbone_bridge;
- **Display name:** Avalon MM to Wishbone Bridge;
- **Group:** LAB-PDS;
- **Description:** This is an interface for exchanging data between an Avalon MM Master and a Wishbone Slave;
- **Created by:** joserodrigues.oliveiraneto@ufpe.br.

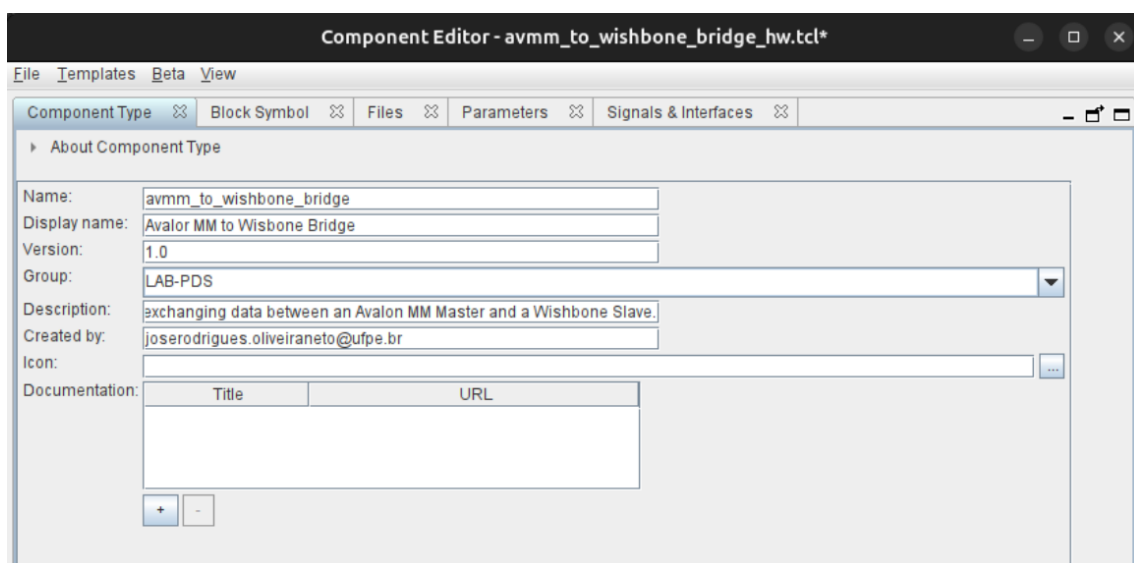


Figura 4: Tela **Component Editor** do Platform Designer destacando os campos a serem preenchidos na aba **Component Type**.

3. Não mexeremos na Aba **Block Symbol**, logo, é possível dar **Next** até chegar a aba **Files**. Neta aba, selecionar o arquivo do *IP core* a ser criado. Para isto, clique em **Add File**, destacado em vermelho na Figura 5, e caminhe até aonde está o arquivo (/hdl_files/utills). Após selecionar o arquivo, clicar em **Analyze Synthesis Files**, destacado em azul na Figura 5.

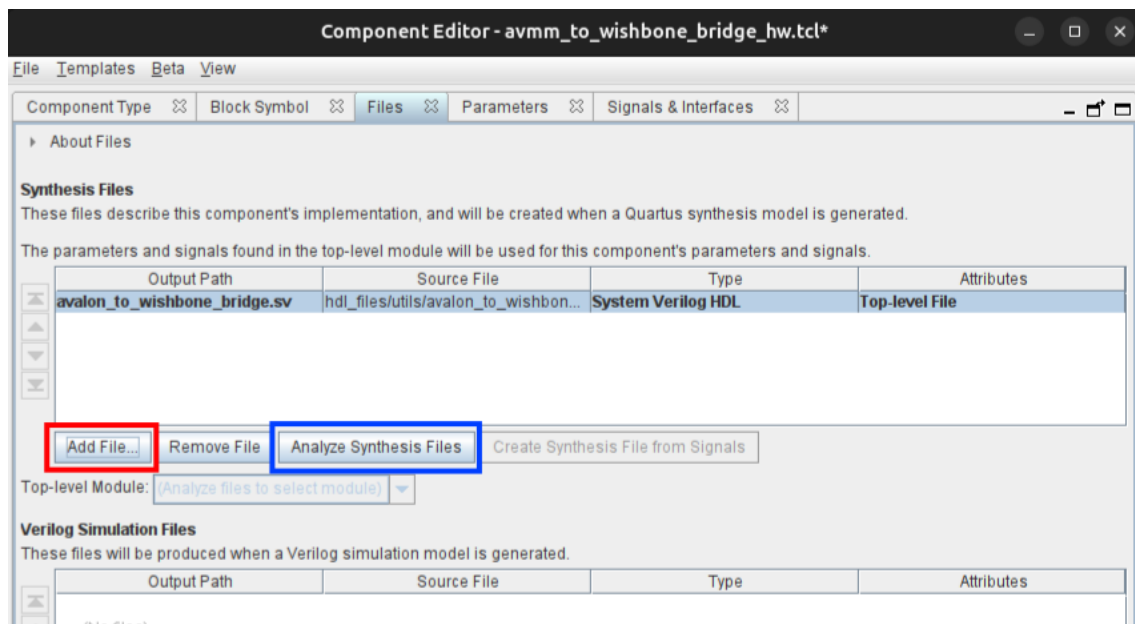


Figura 5: Tela **Componet Editor** do Platform Designer destacando os campos a serem preenchidos na aba **Files**.

4. Será aberta uma janela com a análise dos arquivos adicionados ao *IP core*. É esperado que não tenha erros de sintaxe, como mostrado na Figura 6. Embora ainda existirão erros relacionados aos sinais, que vamos corrigir em seguida.

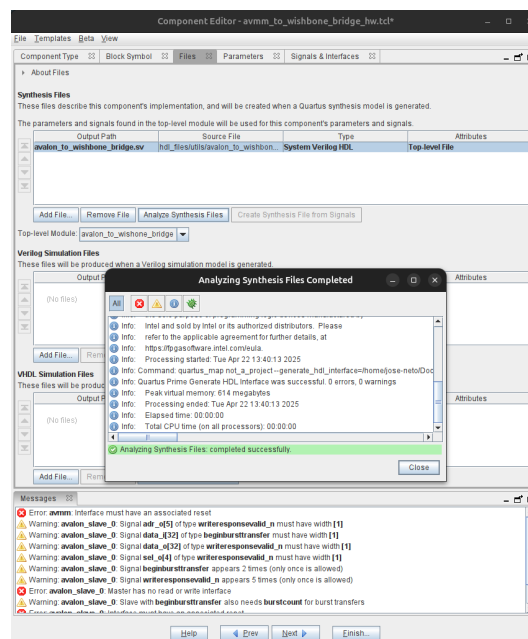


Figura 6: Tela **Componet Editor** do Platform Designer na aba **Files**, com a janela **Analyzing Synthesis Files Completed** em destaque.

5. Após fechar a janela **Analyzing Synthesis Files Completed**, não mexeremos na aba **Parameters**. Devemos então seguir para aba **Signals & Interfaces**. O Platform Designer tentará agrupar os sinais em barramentos que ele já possui. Será possível ver que ele conseguiu identificar um dos barramentos Av-MM quase com perfeição. Mas, reforçando a semelhança entre os barramentos, ele tentará associar o barramento Wishbone como um barramento Av-MM. Será necessário fazer as seguintes modificações:

- (a) Arraste o sinal `clk_i` para interface **clock** e o sinal `rst_i` para interface **reset**, como mostrado na Figura 7.

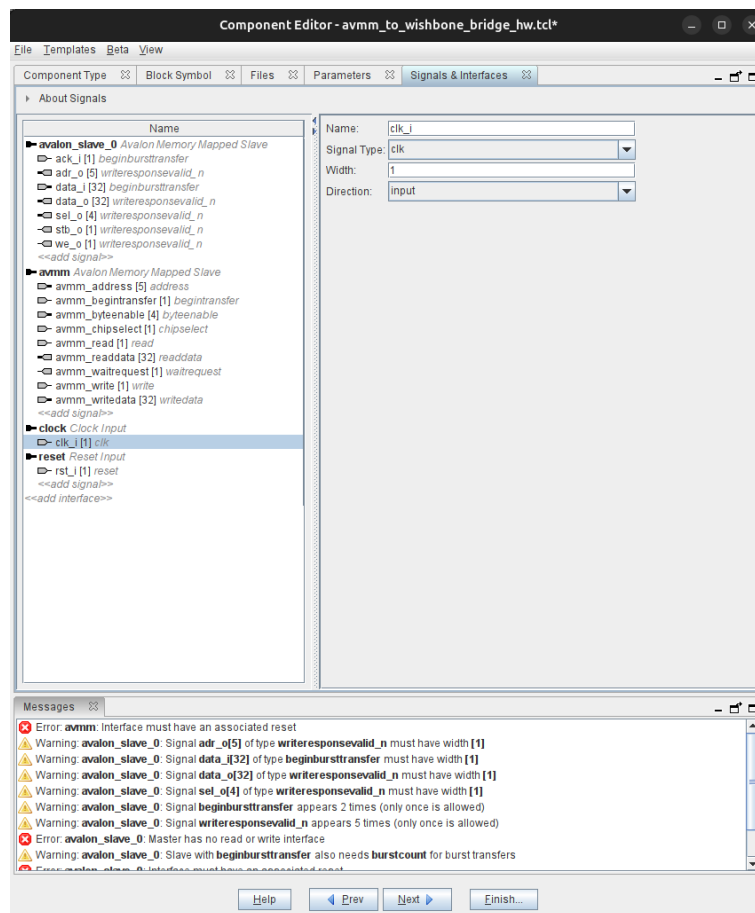


Figura 7: Tela **Component Editor** do Platform Designer na aba **Signals & Interfaces**.

- (b) Clicando sobre o nome **avmm** na aba **Name**, será aberto as características do barramento a direita da janela. Configurar como mostrado na Figura 8.

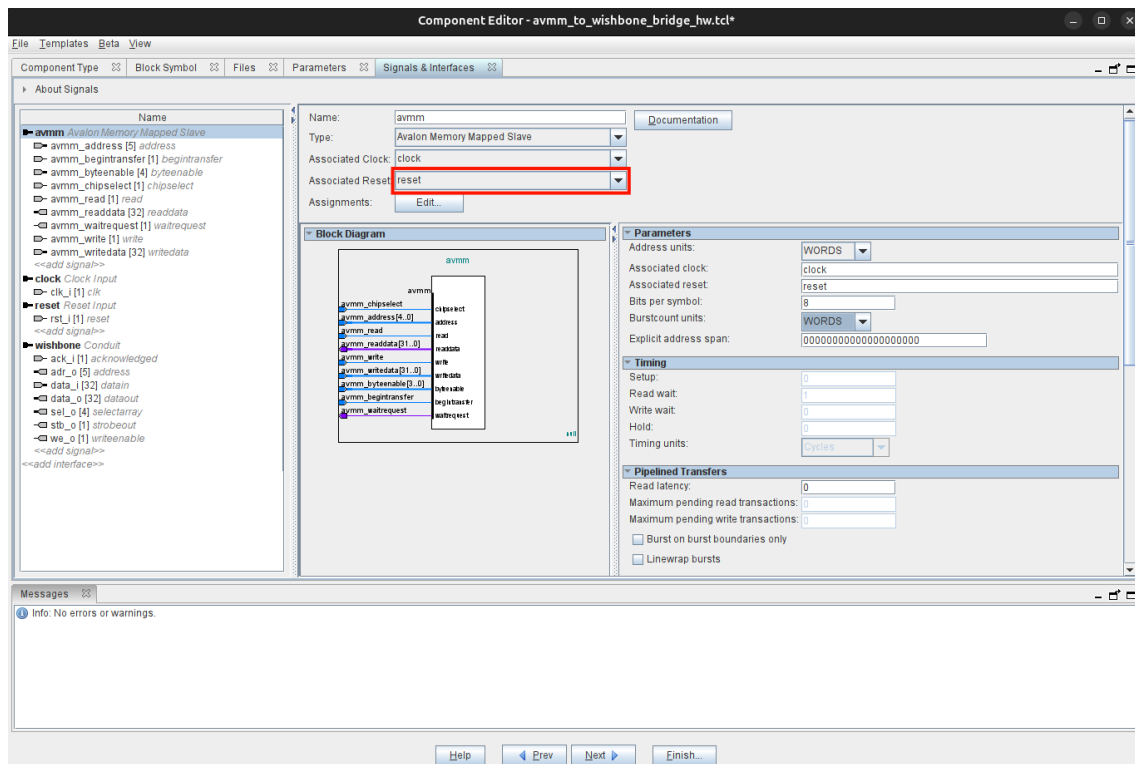


Figura 8: Tela Componet Editor do Platform Designer na aba **Signals & Interfaces**, com destaque na interface **avmm**.

(c) Para a interface **avalon_slave_0**, será necessário mudar vários sinais. No final, a interface deverá ficar como mostrado na Figura 9.

- **Name:** wishbone;
- **Type:** Conduit;
- **Associated Clock:** clock;
- **Associated Reset:** reset.

(d) Para cada um dos sinais do barramento Wishbone será necessário modificar o **Signal Type**, clicando em cada um e editando:

- **ack_i:** acknowledged;
- **adr_o:** address;
- **data_i:** datain;
- **data_o:** dataout;
- **sel_o:** selectarray;
- **stb_o:** strobeout;
- **we_i:** wwriteenable;

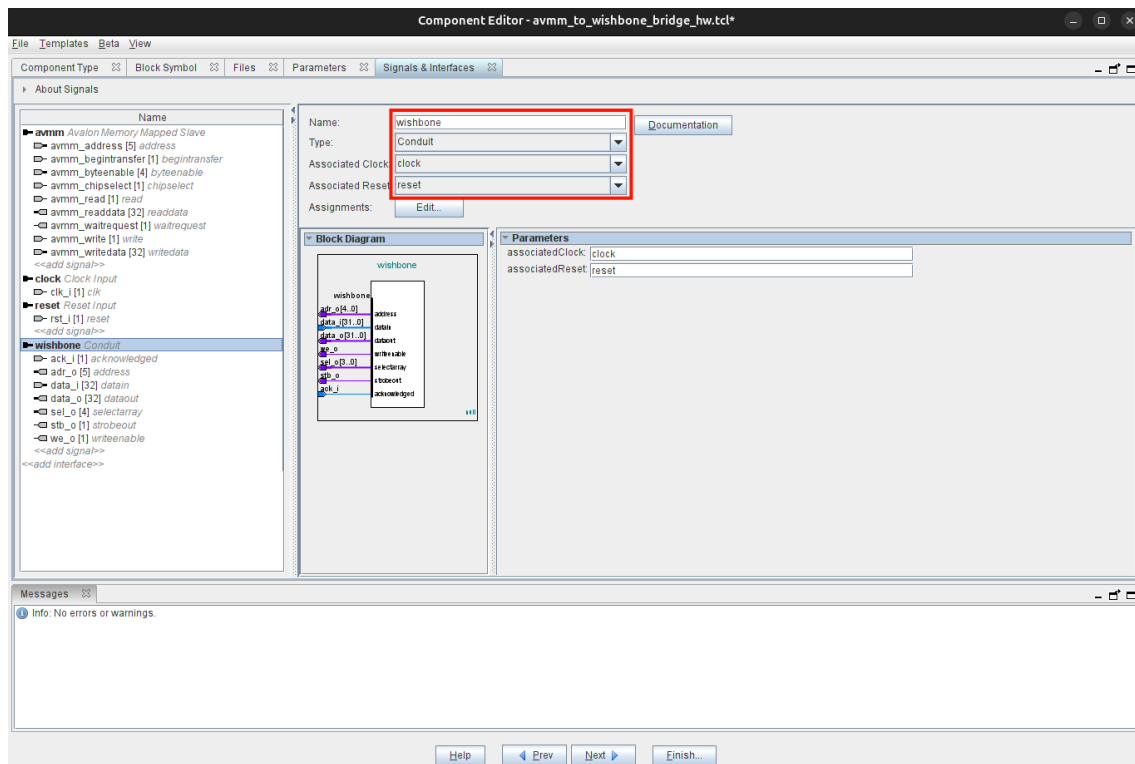


Figura 9: Tela Component Editor do Platform Designer na aba **Signals & Interfaces**, com destaque na interface **avalon_slave_0**.

Referências

- [1] R. Herveille, “WISHBONE system-on-chip (SoC) interconnection architecture for portable IP cores.” online, oct 2019. Release B3.1.
- [2] M. Sharma, “Wishbone bus architecture - a survey and comparison,” *Int. j. VLSI des. commun. syst.*, vol. 3, pp. 107–124, Apr. 2012.
- [3] Altera, “Avalon memory-mapped interface - specification.” online, 2006. Disponível em: https://www.cs.columbia.edu/~sedwards/classes/2007/4840/mnl_avalon_spec.pdf.